Análisis teórico de C1, C2 y C3: avance de la sesión 2

Grupo 3: alineamiento de lecturas genómicas con k-mers como semillas.

0. Contexto: dónde entra cada componente

```
genoma (n bases) ──► k-mers (kmer en 2 bits, posición) ──► C1 merge sort
──► índice de semillas

(claves ordenadas +

C2 posiciones por k-mer)
lectura (m bases) ──► sus k-mers ──► búsqueda binaria en el índice ──►
votos por posición inicial
                  ──► región candidata (m + 2·margen bases) ──► C3 Smith-
Waterman ──► alineamiento
```

- **Codificación** (comun/kmers.py): A=0, C=1, G=2, T=3, y el k-mer se lee como un número en base 4. Un k-mer con k ≤ 31 cabe en 64 bits, comparar dos k-mers cuesta O(1) y el orden de los enteros coincide con el orden alfabético. Una ventana deslizante extrae los n−k+1 k-mers en Θ(n).
- **Índice** (comun/indice.py): se construye en Θ(n log n), dominado por C1, y responde cada consulta con búsqueda binaria en Θ(log u), donde u es el número de k-mers distintos.
- **Alineador** (comun/alineador.py): Smith-Waterman alinea cada lectura solo contra la región candidata, en Θ(m·(m+2·margen)) en vez de Θ(m·n) contra el genoma completo. Esa diferencia es la razón de usar semillas.

C1. Merge sort de k-mers

Código: c1_mergesort/merge_sort.py.

PSEUDOCÓDIGO

```
MERGE-SORT(A):
  si |A| ≤ 1: devolver A
  mid ← ⌊|A|/2⌋
  I ← MERGE-SORT(A[0..mid))
R.agregar(I[i]); i++
  D ← MERGE-SORT(A[mid..|A|))
R.agregar(D[j]); j++
  devolver MEZCLAR(I, D)
```

```
MEZCLAR(I, D):
  R ← [], i ← 0, j ← 0
   mientras i < |I| y j < |D|:
      si I[i] ≤ D[j]:

       si no:

   agregar el resto de I y de D a R
   devolver R
```

RECURRENCIA LEÍDA DEL CÓDIGO

Línea de `merge_sort`	Costo
`if len(a) <= 1: return a`	Θ(1)
`a[:mid]`, `a[mid:]` (copiar las mitades)	Θ(n)
dos llamadas recursivas sobre ⌊n/2⌋ y ⌈n/2⌉ elementos	T(⌊n/2⌋) + T(⌈n/2⌉)
`mezclar`: cada iteración agrega un elemento a `res`	Θ(n)

```
T(n) = T(⌊n/2⌋) + T(⌈n/2⌉) + θ(n),     T(1) = θ(1)
```

que para el análisis asintótico se escribe T(n) = 2T(n/2) + Θ(n). Los pisos y techos no cambian el orden.

TEOREMA MAESTRO

Con a = 2 (subproblemas), b = 2 (factor de reducción) y f(n) = Θ(n):

- n^(log_b a) = n^($\log_2$ 2) = n.
- f(n) = Θ(n) = Θ(n^(log_b a)): es el **caso 2**. El trabajo es el mismo en cada uno de los $\log_2$ n niveles del árbol de recursión.
- Por tanto **T(n) = Θ(n^(log_b a) · log n) = Θ(n log n)**.

Comprobación con el árbol de recursión: el nivel i tiene 2^i subproblemas de tamaño $n/2^i$ y cuesta $c \cdot n$ en total. Hay $\log_2 n + 1$ niveles, así que $T(n) = c \cdot n \cdot (\log_2 n + 1) = \Theta(n \log n)$.

MEJOR, PEOR Y CASO ESPERADO

El tiempo es **Θ(n log n) en los tres casos**: las copias de `a[:mid]` y de `res` cuestan Θ(n) en cada nivel sin importar los datos. Lo que sí depende de los datos es el número de **comparaciones** C(n). Para n potencia de 2:

Caso	Datos que lo provocan	Comparaciones por mezcla de tamaño s	C(n)
Mejor	arreglo ya ordenado (o en orden inverso): una mitad se agota primero	s/2	$(n/2) \cdot \log_2 n$
Peor	permutación en la que las mitades se intercalan en cada mezcla (p. ej. 0,4,2,6,1,5,3,7)	s − 1	$n \cdot \log_2 n - n + 1$
Esperado	permutación aleatoria	≈ s − 2	$\approx n \cdot \log_2 n - 1{,}25 \cdot n$

En nuestro caso hay muchos k-mers repetidos, pero ordenamos pares (kmer, posición) y la posición es única, así que todas las claves son distintas y el análisis aplica sin cambios.

- **Memoria extra:** Θ(n) por las listas auxiliares, más una pila de recursión de profundidad $\lceil \log_2 n \rceil$.
- **Estabilidad:** el `<=` de `mezclar` conserva el orden relativo de los elementos iguales.

COTA INFERIOR Ω(N LOG N): ÁRBOL DE DECISIÓN

Todo algoritmo que ordena **solo comparando** pares de elementos se representa con un árbol de decisión binario:

- cada nodo interno es una comparación `x ≤ y` y tiene dos hijos (sí / no);
- cada hoja es una permutación de salida;

- una ejecución es un camino de la raíz a una hoja, y su longitud es el número de comparaciones.

1. Para ordenar bien cualquier entrada de n claves distintas, cada una de las **n!** permutaciones tiene que aparecer en al menos una hoja.
2. Un árbol binario de altura h tiene a lo sumo 2^h hojas. Entonces $2^h \geq n!$, es decir **$h \geq \log_2(n!)$**.
3. n! ≥ (n/2)^(n/2), porque los n/2 factores mayores valen al menos n/2. Entonces $\log_2(n!) \geq (n/2)\cdot\log_2(n/2) = \Omega(n \log n)$. Por Stirling, $\log_2 n! = n \log_2 n - n \log_2 e + O(\log n) \approx n \log_2 n - 1{,}44 \cdot n$.

Por tanto **todo ordenamiento por comparaciones hace Ω(n log n) comparaciones en el peor caso**. Merge sort hace $n \log_2 n - n + 1$ en el peor caso, así que es **asintóticamente óptimo**: solo se aleja de la cota en ≈ 0,44·n.

La cota también vale para el **caso promedio**: la profundidad media de las hojas de un árbol binario con n! hojas es ≥ $\log_2 n!$. Pero **no vale para cada entrada**: con la entrada ordenada merge sort hace $(n/2)\cdot\log_2 n < \log_2 n!$. El test `test_mejor_caso_queda_bajo_log2_factorial` lo verifica.

C2. Arreglo dinámico de posiciones (duplicación)

Código: c2_arreglo_dinamico/arreglo_dinamico.py. En el índice, cada k-mer distinto tiene un arreglo con las posiciones donde aparece en el genoma.

OPERACIÓN Y COSTO REAL

`agregar(x)`: si n = capacidad, se reserva un arreglo de capacidad 2·c y se **copian** los n elementos; después se **escribe** x. Modelo de costo:

- 1 por escribir el elemento nuevo;
- 1 por cada elemento copiado al duplicar.

El append número i cuesta i si i−1 es potencia de 2 (hay que duplicar) y 1 en otro caso. El peor caso de una sola operación es Θ(n).

MÉTODO CONTABLE

Se cobran **ĉ = 3 créditos por append**:

Créditos	Uso
1	paga la escritura del elemento nuevo
2	quedan depositados “sobre” ese elemento

Invariante: en todo momento, el saldo del banco es ≥ 0 y alcanza para la siguiente duplicación.

Prueba. Supongamos que el arreglo pasa de capacidad c a 2c, lo que ocurre al insertar el elemento c+1, y hay que copiar c elementos.

- La duplicación anterior fue de c/2 a c. Desde entonces se insertaron c/2 elementos y cada uno dejó 2 créditos: **c créditos**, exactamente lo que cuesta copiar los c elementos.
- Con capacidad inicial 1, la primera duplicación (1 → 2) copia 1 elemento, pagado con los 2 créditos del primer append.

Por inducción sobre las duplicaciones, el saldo nunca es negativo. Como el saldo es créditos cobrados menos costo real y es ≥ 0:

```
costo real (n appends) ≤ 3n   ⇒   costo amortizado por append = O(1)
```

Cálculo exacto: costo real = n + (suma de 2^j para todo $2^j < n$) = n + ($2^{\lceil \log_2 n \rceil} - 1$) < 3n. La razón costo/n se acerca a 3 justo después de una duplicación ($n = 2^j + 1$) y a 2 justo antes ($n = 2^j$).

Verificación en el código: la clase lleva el banco en `saldo` y su mínimo en `saldo_minimo`. El test `test_saldo_de_creditos_nunca_negativo` comprueba saldo ≥ 0 hasta $n = 10^5$.

COTAS

Operación	Peor caso	Amortizado	Mejor caso
`agregar`	Θ(n) (duplica)	Θ(1)	Θ(1)
`a[i]`	Θ(1)	Θ(1)	Θ(1)
n appends	Θ(n)		

Memoria: capacidad < 2n, es decir Θ(n).

C3. Smith-Waterman (alineamiento local por programación dinámica)

Código: c3_smith_waterman/smith_waterman.py.

PROBLEMA

Dada una lectura a (largo m) y una región b (largo n), se busca el par de subcadenas a[i'..i) y b[j'..j) cuyo alineamiento tiene **puntaje máximo**. Los puntajes por columna son:

Columna	Puntaje
bases iguales	+2
bases distintas	−1
hueco (inserción o borrado)	−2

Que sea **local** importa porque la región candidata incluye bases de más a los lados (el margen) y la lectura puede traer errores en las puntas. Por eso no se alinean las cadenas completas.

RECURRENCIA

H[i][j] es el mejor puntaje de un alineamiento local que **termina** exactamente en a[i−1] y b[j−1] (o vacío).

$$H[i][0] = H[0][j] = 0$$

$$H[i][j] = \max \begin{cases} 0 & \text{empezar de cero} \\ H[i-1][j-1] + s(a_{i-1}, b_{j-1}) & \text{alinear las dos bases} \\ H[i-1][j] - 2 & a_{i-1} \text{ contra un hueco} \\ H[i][j-1] - 2 & b_{j-1} \text{ contra un hueco} \end{cases}$$

$$s(x, y) = +2 \text{ si } x = y, \quad -1 \text{ si } x \neq y$$

Respuesta: el máximo de H[i][j] sobre todas las celdas (i, j). El alineamiento se reconstruye retrocediendo desde esa celda: en cada paso se elige la celda de la que vino el valor, hasta llegar a un 0.

SUBESTRUCTURA ÓPTIMA

Sea A un alineamiento óptimo que termina en (i, j) con puntaje H[i][j] > 0. Su última columna tiene que ser una de tres:

1. (a_{i-1}, b_{j-1}): lo que queda, A', termina en (i−1, j−1) y vale H[i][j] − s.
2. (a_{i-1}, −): A' termina en (i−1, j).
3. (−, b_{j-1}): A' termina en (i, j−1).

Si A' no fuera óptimo para su subproblema, se podría cambiar por uno mejor y pegarle la misma última columna. El resultado sería un alineamiento que termina en (i, j) con puntaje mayor que A: contradicción (**argumento de cortar y pegar**). El 0 cubre el caso en que conviene no extender nada, porque todo lo anterior suma negativo. Así, H[i][j] solo depende de soluciones óptimas de subproblemas más pequeños.

SUBPROBLEMAS SUPERPUESTOS

Una recursión directa sobre (i, j) llama a (i−1, j−1), (i−1, j) e (i, j−1), y cada una de ellas vuelve a llamar a (i−1, j−1). El número de llamadas cumple R(i, j) ≥ 3·R(i−1, j−1), es decir R ≥ 3^min(m,n): **exponencial**. Pero solo hay (m+1)(n+1) subproblemas distintos. La tabla calcula cada uno una vez.

TABLA Y ORDEN DE LLENADO

- Tabla H de tamaño (m+1)×(n+1).

- Fila 0 y columna 0 en 0 (caso base).
- Se llena fila por fila, de izquierda a derecha. Así, al calcular H[i][j] ya están listas las tres celdas de las que depende: la diagonal, la de arriba y la de la izquierda.

COMPLEJIDAD

	Mejor	Peor	Esperado
Llenar la tabla	Θ(mn)	Θ(mn)	Θ(mn)
Retroceso	O(1) (sin coincidencias, puntaje 0)	Θ(m + n)	
Memoria	Θ(mn)	Θ(mn)	Θ(mn)

- **Tiempo:** el tiempo no depende de los datos. Siempre se calculan las m·n celdas, cada una en O(1), y el contador `celdas` lo comprueba (= m·n).
- **Memoria:** si solo se necesitara el puntaje, bastarían dos filas: Θ(n) de memoria con el mismo tiempo. Guardamos la tabla completa porque necesitamos el alineamiento.
- **En el pipeline:** m = 50 y la región mide m + 2·10 = 70, así que cada lectura cuesta Θ(m^2) ≈ 3500 celdas. Contra el genoma completo serían m·n = $50 \cdot 10^6$ celdas.

CORRECCIÓN VERIFICADA CON PRUEBAS

tests/test_c3_smith_waterman.py comprueba:

- **Fuerza bruta:** en 40 pares aleatorios de largo ≤ 6, el puntaje coincide con un oráculo que prueba el alineamiento global (Needleman-Wunsch) entre **todos** los pares de subcadenas. Eso es literalmente la definición de alineamiento local.
- **Reconstrucción:** el alineamiento devuelto tiene el puntaje reportado y, quitando los huecos, coincide con las subcadenas a[inicio_a:fin_a] y b[inicio_b:fin_b].
- **Ejemplo clásico de la literatura** (TGTTACGG contra GGTTGACTA con +3/−3/−2): puntaje 13, alineamiento GTT-AC / GTTGAC.
- **Lecturas sintéticas** con 3 % de sustituciones y 2 % de indels: SW las ubica a ≤ 3 bases de su posición real.